

TESTING

RAG (Retrieval Augmented Generation)

LLMs are constrained by a fundamental architecture limit : the context window. Even with longer context, the performance of the model deteriorates, leading to inability to accurately interpret and reason over long, complex , domain-specific documents. To mitigate the limitation, RAG emerged as the dominant solution.

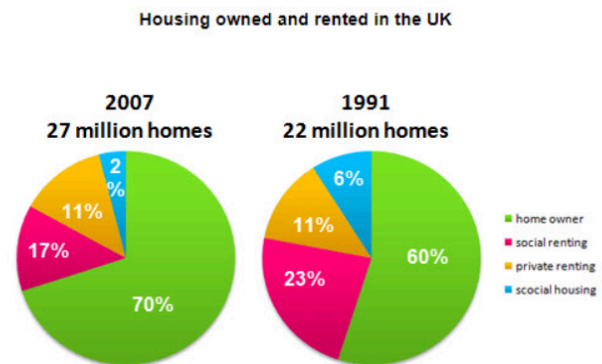
RAG (Retrieval Augmented Generation) is an AI framework that enhances the capabilities of LLMs by combining text generation with information retrieval from the external knowledge base. Unlike traditional LLM that solely rely on pre-trained data, RAG fetches relevant data in real time generating more accurate and contextually aware responses.

Vector DB contains embeddings of knowledge base and retrieves the most similar instance to the query vector. It then feeds the procured data to a LLM which ultimately generates response. The LLM does not need to be retrained as it can fetch context sensitive data from the updated knowledge base. RAG ensures that the generated response displays its source, which helps to check its validity and eradicate any hallucinations.

RAG allows LLMs to identify whether the query is unanswerable (Not retrieved by RAG) instead of providing irrelevant information. RAG can hinder the user experience due its incompetence, by not

answering the query instead of procuring the best of the available information.

The figure below depicts the working of the system:



Tree Search

Documentation

[:https://docs.pageindex.ai/tutorials/tree-search](https://docs.pageindex.ai/tutorials/tree-search)

- **LLM Tree Search**

One can simply provide their LLM with a prompt, to perform traversal.

(PageIndex uses a technique which combines the LLM tree search and value function-based Monte Carlo Tree Search ([MCTS](#))) [Link : [Monte Carlo tree search - Wikipedia](#)]

User preferences and Expert Knowledge can be incorporated by tweaking the LLM tree prompt unlike traditional RAGs which require fine-tuning.

Scrap out the user expert knowledge snippets from the database with domain

specific rules. This can be done by keyword matching, semantic similarity or LLM based search. Integrate it with the LLM Tree search prompt to achieve more specific and targeted retrieval.

Expert Preference:

If the query mentions EBITDA adjustments, prioritize Item 7 (MD&A) and footnotes in Item 8 (Financial Statements) in 10-K reports.

- **Value-based Tree Search**

An AI model predicts a value that represents the similarity of content with the user query. Unlike the classic RAG approach, here we parent nodes based on association and not the chunks, itself.

1. Each node is divided into several chunks
2. We perform vector search to retrieve top-k elements
3. The parent node is scored based on associativity of retrieved chunks.

(NodeScore formula is given in the documentation)

The scoring favours nodes with fewer highly, relevant chunks over those with many weakly relevant ones

a. DRAWBACKS

1. LLM-based search may be slower due to the need for LLM reasoning.
2. Relying solely on summaries may result in loss of important details.
3. Using a pre-trained embedding model to initialize the value function, may miss the relevant nodes and struggle to integrate expert knowledge.

Hybrid Tree Search

Inspired by Google's AlphaGo (Link : [AlphaGo — Google DeepMind](#))

The hybrid model combines to leverage strengths and overcome the limitations of both the approaches mentioned above.

Process :

1. Perform value-based tree search and LLM-based tree search simultaneously.
2. Maintain a queue of unique node elements. As nodes are returned from either search, add them to the queue only if they are not already present.
3. A consumer processes nodes from the queue, extracting or summarizing relevant information from each node.
4. The agent continually evaluates whether enough information has been gathered. If so, the process can terminate early.
5. Do consider the first page of the next node, to make sure the content is not lost.
6. Can use OPEN AI text embeddings.

INFERENCE

The following observations were done with the perspective of an end user. The conclusions were drawn by considering following parameters : **Performance, Relevancy to Document, Transparency, Ability to summarize.**

1. GPT - 5.3

OpenAI's model GPT - 5.3 model is a very capable LLM for daily use, but fails miserably in task specific actions. The responses were fairly quick but very superficial in nature. The generated output refers to the external links other than the provided document while not tagging specific parts of the document. Hence, as a consumer it is not advisable to use GPT - 5.3, to handle any domain specific documents.

2. Gemini - 3

Gemini AI developed by Google was the quickest of all the three models chosen. It provides very balanced responses, which were quick as well as refrained to the provided document. Gemini did not tag the contents of the report and provided a basic summary. Gemini - 3 is preferable over GPT - 5.3 for summarizing large documents on a daily basis.

3. PageIndex

PageIndex was the slowest responding model of the three. On an average, it took 3 times longer to generate a response for a 134 pages long document. The response time may rise exponentially as the size of the document is increased. PageIndex did produce the output I was looking for, along with specifying the page number. The model included all the relevant information across multiple pages and provided an elaborate summary in comparison to Gemini.

PageIndex Tree Index Example :

Node {

node_id: string, // Unique node identifier
name: string, // Human-readable label or title
description: string, // Optional detailed explanation of the node
metadata: object, // Arbitrary key-value pairs for context or attributes
sub_nodes: [Node] // Array of child nodes (recursive structure)

}

{

"node_id": "0006",
"title": "Financial Stability",
"start_index": 21,
"end_index": 22,
"summary": "The Federal Reserve ...",
"sub_nodes": [
{
"node_id": "0007",
"title": "Monitoring Financial Vulnerabilities",
"start_index": 22,
"end_index": 28,
"summary": "The Federal Reserve's monitoring ..."

}] }

Roneys Super Car

